研艺士
YANZHISHI

高级算法和数据结构

主讲人：灵灵老师

计算机考研

目录

1. 动态规划

2. 最大连续子序列和

3. 最长不下降子序列

4. 01背包问题

研学士 YANZHISHI

动态规划

1. 什么是动态规划?

动态规划(Dynamic Programming，DP)是一种用来解决一类**最优化**问题的算法思想。简单来说，动态规划将一个复杂的问题分解成若干个子问题，通过综合**子问题的最优解**来得到**原问题的最优解。**（动态规划会将每个求解过的子问题的解记录下来，这样当下一次碰到同样的子问题时，就可以直接使用之前记录的结果，而**不用重复计算。**）

2. 动态规划实现方法

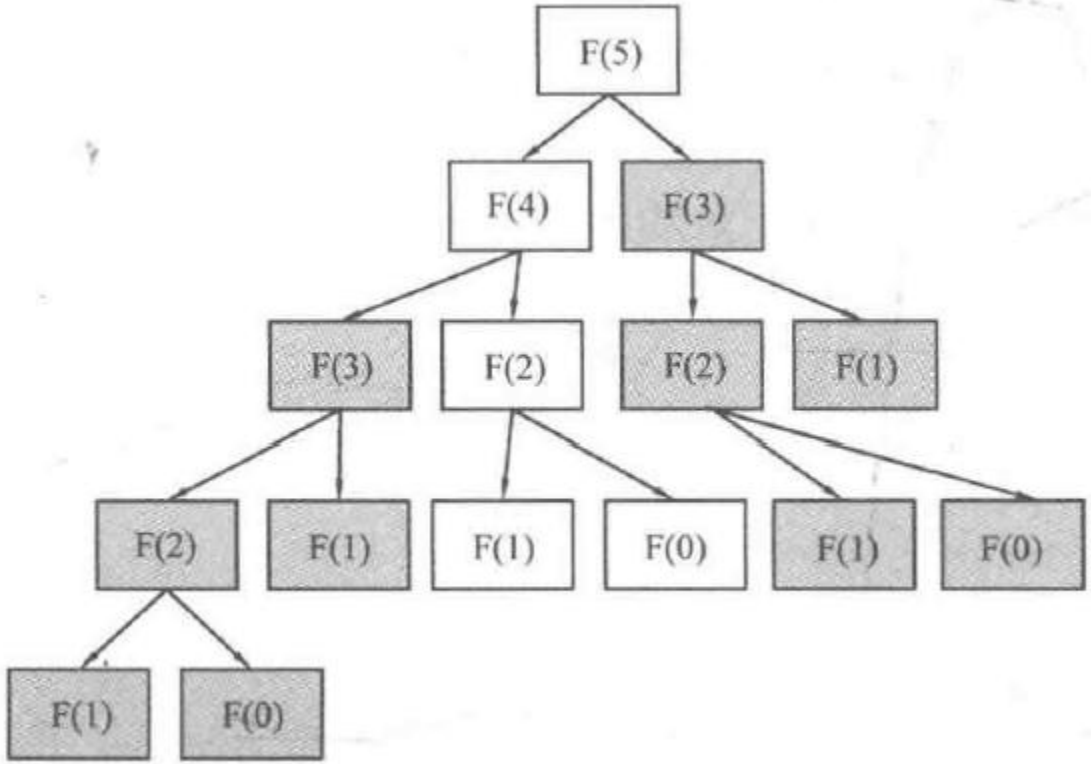
一般可以使用**递归**或者**递推**的写法来实现动态规划，其中递归写法又称作记忆化搜索。

研之士 YANZHISHI

动态规划-递归写法

例：斐波那契数列

```
int F(int n) {
    if(n == 0 || n == 1) return 1;
    else return F(n-1) + F(n-2);
}
```



$T(n)=O(2^n)$，重叠子问题

缺点：没有及时保存中间计算的结果，**会重复计算子问题的解**，基本不能承受n较大的情况。

动态规划-递归写法

◆ 怎么避免重复计算?

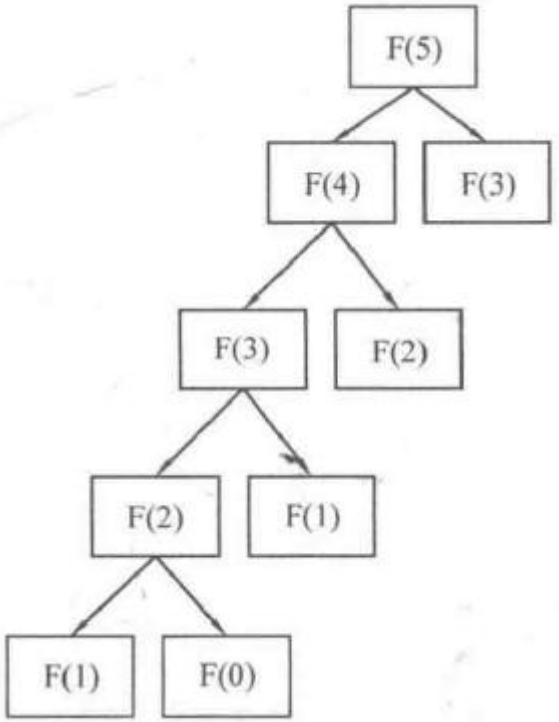
① 可以用一个一维数组 dp，来保存已经计算过的结果，其中**dp[i]存储 F(i)的结果**，并用 dp[i]=-1 表示 F(i)当前还没有被计算过。

② 在递归中加入一个判断，判断dp[i]是否是-1:

如果不是-1，说明已经计算过F(i)，不需要重复计算，直接返回 dp[i]即可;

否则，按照递归式进行递归。

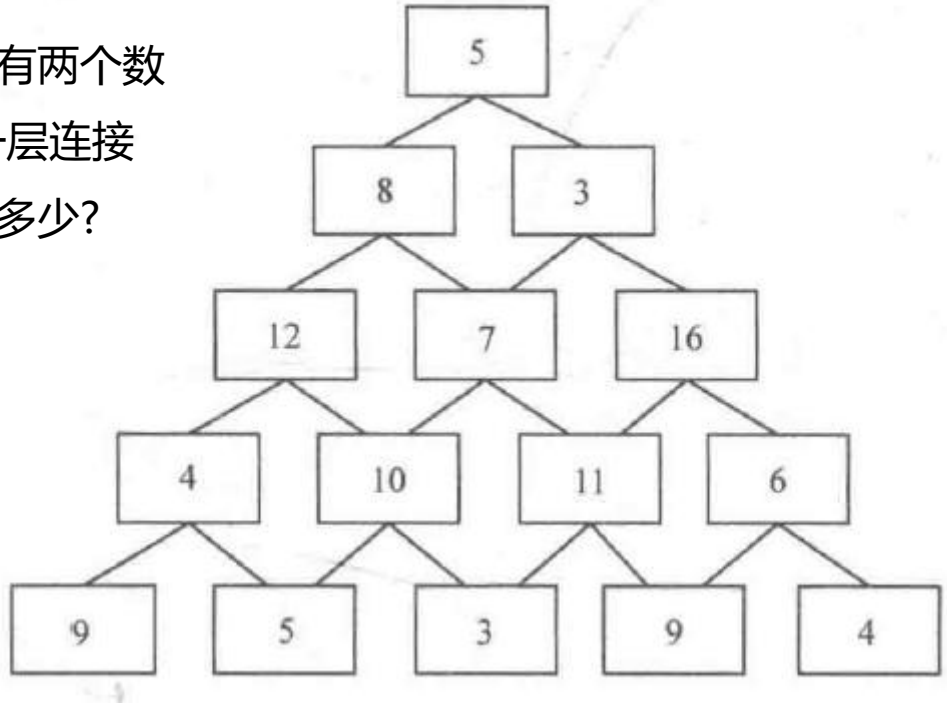
```
int F(int n) {
    if(n == 0 || n == 1) return 1;  //递归边界
    if(dp[n] != -1) return dp[n];  //已经计算过，直接返回结果，不再重复计算
    else {
        dp[n] = F(n-1) + F(n-2);  //计算 F(n)，并保存至 dp[n]
        return dp[n];  //返回 F(n)的结果
    }
}
```



动态规划-递推写法

例： 数塔问题

如图所示，将一些数字排成数塔的形状，其中第一层有一个数字，第二层有两个数字.....第n层有n个数字。现在要从第一层走到第n层，每次只能走向下一层连接的两个数字中的一个，问：最后将路径上所有数字相加后得到的和最大是多少?



（递归）

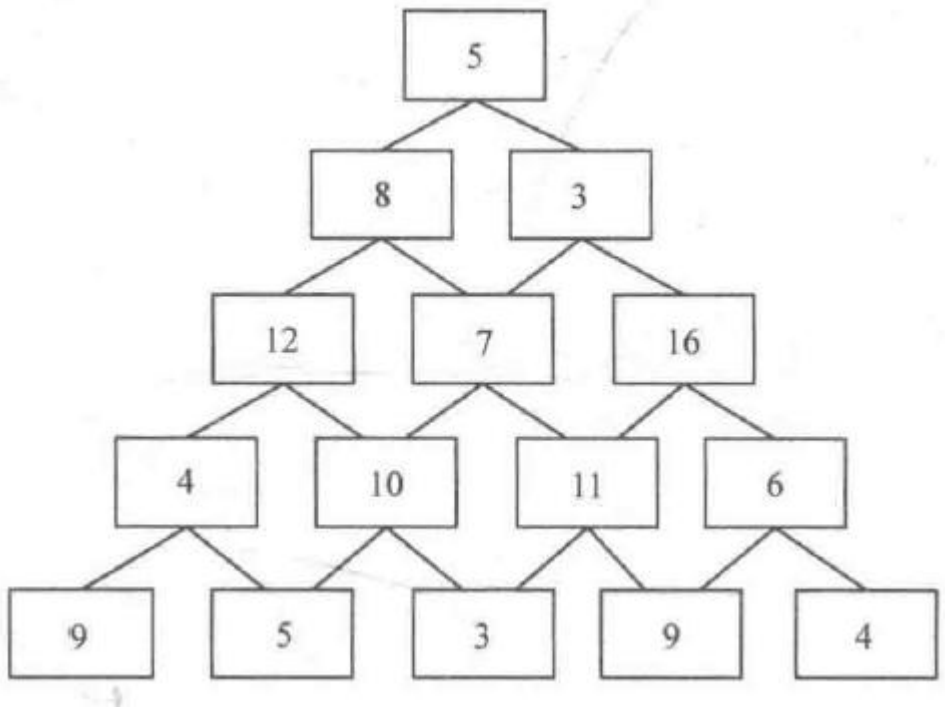
```
int f[N][N];//二维数组
int F(int n, int i, int j){//树的两个分叉
    if(i == n)              //到最底层，那么该元素就是最大值
        return f[n][j];
    else            //否则选子树上到底层距离的较大数加上自身return
max(F(n, i + 1, j),F(n, i+1,j+1)) + f[i][j];
}
```

动态规划-递推写法

而动态规划的**递推写法**：（**自底向上**）

从边界出发，通过状态转移方程扩散到整个dp数组。

$$dp[i][j] = \max(dp[i+1][j], dp[i+1][j+1]) + f[i][j]$$



把 dp[i][j]称为问题的**状态**，而把上面的式子称作**状态转移方程**，

它把状态 dp[i][j]**转移为**dp[i+1][i]和 dp[i+1][j+1]。

那么，什么时候会到头呢?

数塔的最后一层的dp值总是等于元素本身，

即 dp[n][j] ==f[n][j]，这种可以直接确定其结果的部分称为**边界**，

动态规划-递推写法

```
for(int i = 1; i <= n; i++) {
    for(int j = 1; j <= i; j++) {
        scanf("%d", &f[i][j]);  //输入数塔
    }
}
//边界
for(int j = 1; j <= n; j++) {
    dp[n][j] = f[n][j];
}
//从第 n-1 层不断往上计算出 dp[i][j]
for(int i = n - 1; i >= 1; i--) {
    for(int j = 1; j <= i; j++) {
        //状态转移方程
        dp[i][j] = max(dp[i + 1][j], dp[i + 1][j + 1]) + f[i][j];
    }
}
```

总结：动态规划

① **重叠子问题**

② **最优子结构**

动态规划会将每个求解过的子问题的解记录下来，避免重复计算。

最优子结构保证了动态规划中原问题的最优解可以由子问题的最优解推导而来。

最大连续子序列和

给定一个数字序列 $A_1, A_2, \cdots, A_n$，求 i, j ($1 \leqslant i \leqslant j \leqslant n$)，使得 $A_i + \cdots + A_j$ 最大，输出这个最大和。

样例：

–2 11 –4 13 –5 –2

显然 11+(–4)+13=20 为和最大的选取情况，因此最大和为 20

动态规划（递推）方法：

令状态 dp[i]表示**以 A[i]作为末尾**的连续序列的最大和，那么要求的最大和其实就是 dp[0],dp[1],...,dp[n-1]中的最大值。（如何求解 dp 数组？**状态转移方程**）

① 这个最大和的连续序列只有一个元素，即以 A[i]开始，以 A[i]结尾。

② 这个最大和的连续序列有多个元素，即从前面某处 A[p]开始(p<i)，一直到 A[i]结尾。

$$dp[i] = \max\{A[i], dp[i-1] + A[i]\}$$

这个式子**只和i与i之前的元素有关**，且**边界**为 dp[0]=A[0] 。

最大连续子序列和

```
//边界
dp[0] = A[0];
for(int i = 1; i < n; i++){
    //状态转移方程
    dp[i] = max(A[i], dp[i - 1] + A[i]);
}
//dp[i]存放以A[i]结尾的连续序列的最大和，需要遍历i得到最大的才是结果
int k = 0;
for(int i = 1; i < n; i++){
    if(dp[i] > dp[k]){
        k = i;
    }
}
printf("%d\n", dp[k]);
```

最大连续子序列和

KY21 最大序列和_牛客

描述

给出一个整数序列S，其中有N个数，定义其中一个非空连续子序列T中所有数的和为T的"序列和"。 对于S的所有非空连续子序列T，求最大的序列和。 变量条件：N为正整数，N≤1000000，结果序列和在范围（-2^63,2^63-1）以内。

输入描述：

第一行为一个正整数N，第二行为N个整数，表示序列中的数。

输出描述：

输入可能包括多组数据，对于每一组输入数据， 仅输出一个数，表示最大序列和。

示例1

输入：5 复制

```
1 5 -3 2 4

6
1 -2 3 4 -10 6

4
-3 -1 -2 -5
```

输出：9 复制

```
7
-1
```

研芝士 YANZHISHI

最长不下降子序列

最长不下降子序列（Longest Increasing Sequence，LIS）是这样一个问题：

在一个数字序列中，找到一个最长的子序列（可以不连续），使得这个子序列是不下降（非递减）的。

例如，现有序列 A = {1, 2, 3, –1, –2, 7, 9}（下标从 1 开始），它的最长不下降子序列是{1, 2, 3, 7, 9}，长度为 5。另外，还有一些子序列是不下降子序列，比如{1, 2, 3}、{–2, 7, 9}等，但都不是最长的。

枚举：对于每个元素有取和不取两种选择，然后判断序列是否为不下降序列。如果是不下降序列，则更新最大长度，直到枚举完所有情况并得到最大长度。

动态规划（递推）方法：

令状态 dp[i]表示**以 A[i]作为末尾**的最长不下降子序列长度，那么要求的结果其实就是 dp[0],dp[1],...,dp[n-1]中的最大值。（如何求解 dp 数组？**状态转移方程**）

最长不下降子序列

例如，现有序列 A = {1, 2, 3, –1, –2, 7, 9}（下标从 1 开始），它的最长不下降子序列是{1, 2, 3, 7, 9}，长度为 5。另外，还有一些子序列是不下降子序列，比如{1, 2, 3}、{–2, 7, 9}等，但都不是最长的。

① 如果存在 A[i]之前的元素 A[j] (j < i)，使得 A[j]≤A[i]且 dp[j] + 1> dp[i]

② 如果 A[i]之前的元素都比 A[i]大，那么 A[i]就只好自己形成一条 LIS，

$$dp[i] = \max\{1, dp[j]+1\}$$
$$(j = 1, 2, \cdots, i-1 \ \&\& A[j] < A[i])$$

上面的**状态转移方程**中，隐含了**边界**：dp[i]=1(1≤i ≤ n)。

最长不下降子序列

[3698. 搬箱子_AcWing题库]

华华要给厂里进一批新箱子共 n 个，编号为 1 到 n，用一个正整数 a_i 来表示编号为 i 的箱子的高度。

现在华华要按照编号从小到大的顺序选出 m 个箱子运到厂房，要确保编号大的箱子比编号小的箱子高。

也就是对于任意的 $i < j$ 有 $a_i < a_j$，那么 m 最大可以是多少呢?

输入格式

第一行是正整数 n，表示 n 个箱子。

第二行 $a_1, a_2 \ldots a_n$ 分别表示编号为 i 的箱子的高度。

输出格式

输出华华最多可以搬运的箱子个数。

数据范围

$1 \le n \le 500$,
$1 \le a_i \le 10000$

输入样例:

```
7
1 7 3 5 9 4 8
```

输出样例:

```
4
```

研学士 YANZHISHI

01背包问题

01 背包问题是这样的：

有 n 件物品，每件物品的重量为 w[i]，价值为 c[i]。现有一个容量为 V 的背包，问如何选取物品放入背包，使得背包内物品的总价值最大。其中每种物品都只有 1 件。

样例：

```
5 8      //n == 5, V == 8
3 5 1 2 2     //w[i]
4 5 2 1 3     //c[i]
```

枚举：对于每个物品都有两种选择，**时间复杂度非常高**

动态规划（递推）方法：

令 dp[i][v]表示前i件物品($1 \le i \le n, 0 \le v \le V$)恰好装入容量为v的背包中所能获得的最大价值。

（如何求解 dp 数组？**状态转移方程**）

01背包问题

```
5 8      //n == 5, V == 8
3 5 1 2 2     //w[i]
4 5 2 1 3     //c[i]
```

考虑对第 i 件物品的选择策略，有两种策略：

① 不放第 i 件物品，那么问题转化为前 i – 1 件物品恰好装入容量为 v 的背包中所能获得的最大价值，也即 dp[i – 1][v]。

② 放第 i 件物品，那么问题转化为前 i – 1 件物品恰好装入容量为 v – w[i]的背包中所能获得的最大价值，也即 dp[i – 1][v – w[i]] + c[i]。

由于只有这两种策略，且要求获得最大价值，因此

$$dp[i][v]=\max\{dp[i-1][v],dp[i-1][v-w[i]]+c[i]\}$$

$$(1\leqslant i\leqslant n, w[i]\leqslant v\leqslant V)$$

01背包问题

[3445. 点菜问题_AcWing题库]

输入格式

输入的第一行有两个整数 C 和 N，C 代表总共能够报销的额度，N 代表能点菜的数目。

接下来的 N 行每行包括两个整数 P_i 和 V_i，分别表示第 i 道菜的价格和评价分数。

输出格式

输出共一行，一个整数，表示在报销额度范围内，所点的菜能够得到的最大评价分数。

数据范围

$1 \le C \le 1000$,
$1 \le N \le 100$,
$1 \le P_i, V_i \le 100$

输入样例:

```
90 4
20 25
30 20
40 50
10 18
```

输出样例:

```
95
```

研艺士 YANZHISHI

课后习题

跳台阶

骨牌

走方格的方案数

不连续1的子串

最长公共子序列

公共子串计算

计算字符串的编辑距离

谢谢聆听！